Basic Model

BasicModel

Basic Model of Cognition

Logo

The basic model of cognition relies on conceptual hyperplanes and perceptual prototypes to synthesize and analyze the input space. It uses a high-dimensional embedding to characterize mental space (as with the transformer architecture) and also integrates symbolic computation. Its three major operations are intersection (which forms percepts from concepts), union (a bidirectional mapping between concepts and percepts), and equality (where symbols are elements that map across perceptual and conceptual domains).

Files

- **SymPercept.py explores using an invertible weight matrix in a single-layer neural network. The matrix can be represented as a rotation followed by an eigenvalue multiplication followed by another rotation: W = R1 @ D @ R2. It compares the error of an invertible matrix as opposed to two non-invertible matrices that are solved by linear regression.**
- **SPNN.py Symmetric Perception Neural Network is a neural network with a single hidden layer. It uses a fully rectified linear unit (FReLU) as an activation function, and performs much better than the SVD above.**
- **SigmaPi SigmaPi demonstrates the SigmaPi network solving the XOR problem. It uses a temperature to add noise during training, rather than random weight initializaation.**
- **BasicModel The basic self**
- **Model Model.Py is a library for BasicModel.py. It also includes scrapings from various previous codes I have written over my lifetime.**

Guiding Principles

- A symmetric perception neural network (SPNN) uses a piecewise linear activation function to simultaneously predict both input and output. By taking advantage of data that may allow surjective

functions in both directions, it attempts to compute a bijective mapping using a single weight matrix.

Dimensionality

- The output dimensionality of the input layer must be equal to the output dimensionality of the perceptual layer, since the conceptual layer operates on both.
- the input dimensionality of the symbolic layer must be equal to the input dimensionality of the perceptual layer, since they both operate on the output of the conceptual layer.
- The input dimensionality of the output layer must be equal to the sum of the output dimensionalities of the symbolic layers.

Equations

Given a weight matrix ($W \in \mathbb{R}^{m \times n}$) and input vector ($x \in \mathbb{R}^n$), the output vector ($y \in \mathbb{R}^m$) is computed as:

For the Sigma layer:

$$y_j = Wx + b$$

$$y_j = b_j + \sum_{i=1}^{n} W_{ji} x_i$$

For the Pi layer:

$$y_j = b_j \cdot \prod_{i=1}^{n} \left(1 + W_{ji} x_i\right)$$

This is not invertible, but if we are allowed twice as many outputs as inputs, we may: do inversion as:

1. Define:
$$s_j := \sum_i \tanh^{-1}(w_{ji} x_i) = Wx$$
2. In forward pass, use:
$$y_j = b_j \cdot \prod_i (1 - \tanh(w_{ji} x_i)), \quad z_j = b_j \cdot \prod_i (1 + \tanh(w_{ji} x_i))$$
3. In reverse pass, compute:
$$\gamma_j = \frac{1}{2} \log\left(\frac{z_j}{y_j}\right) = \sum_i \tanh^{-1}(\tanh(w_{ji} x_i)) = \sum_i w_{ji} x_i \Rightarrow \gamma = Wx$$
4. Finally:
$$x = W^{-1}\gamma$$

Todo:

- Percepts are intersections of concepts, and concepts are unions of percepts. How can we encode this? Vector space ping-pong. Each space is separate, but the activations of each are accessible to the other. Within each space, parts are possible.
- Make sure Layers are using RandomLayer to implement learning, so that regularization is working well.
  - Figure out how temperature integrates with the LR of the Adam optimizer

- make sure Weights (e.g. LVQ vectors) are normalized
- Use an activation function with an adjustable slope
  - the activation function is basically a set of linear weights which are interpreted as certainty
- siamese networks contrastive loss: err(y) and err(y-like-thing) are pushed apart in vector space, perhaps on a hypersphere (i.e. rotated away on a great circle). Implement contrastive loss, $L = (1—y) * D^2 + y * max(0, m—D)^2$
  - L: Contrastive loss for the pair.
  - D: Distance or dissimilarity between the embeddings.
  - y: Label indicating whether the pair is similar (0 for similar, 1 for dissimilar).
  - m: Margin parameter that defines the threshold for dissimilarity.
- LVQ.reverse()
- LM.reverse() to create english words
- Implement perceptual and conceptual parthood, where parthood is established by the < and > operators.
  - Verify that part hierarchies can be transitive
- Implement distance measures
- Incorporate these methods: function tf = isaPart(s1, s2) tf = all(s1 < s2); end function tf = isaWhole(s1, s2) tf = ~Sensation.isaPart(s1, s2); end function s = intersection(s1, s2) s = min(s1, s2); end function s = union(s1, s2) s = max(s1, s2); end
- Implement splitting and joining
  - Maybe at the symbol layer: symbols have (2 percept) embeddings, derived dynamically from concept embeddings.
- Implement attraction and aversion as activation multipliers on percepts
- Action is a function of Err(x). Learning is a function of Err(y)
- Use binary symbol as output for classification tasks
- Examine a cost function that maximizes the normalized entropy at the output layer, perhaps on the symbolic layer
- Have another look at factorizing the error function, especially with respect to symmetric perception:
  - $E(y**2) = E(y1**2) + E(y2**2) + 2 * E(y1) * E(y2)$
  - $E(y**2) = E(y1**2) + E(y2**2)$
- Create an Order or Level by which to index signals.

ETC

$$e^{y_j} = e^{b_j} + \sum_{i=1}^{n} e^{1+W_{ji}x_i}$$

$$\log(e^{y_j}) = \log(e^{b_j} + \sum_{i=1}^{n} e^{1+W_{ji}x_i})$$

$$\log(e^{y_j}) = \log(e^{b_j}) \cdot \log\left(\sum_{i=1}^{n} e^{1+W_{ji}x_i}\right)$$

$$y_j = b_j \cdot \prod_{i=1}^{n} \log(e^{1+W_{ji}x_i})$$

$$y_j = b_j \cdot \prod_{i=1}^{n} (1 + W_{ji}x_i)$$

Now, is there a way to write $e^{1+W_{ji}x_i}$ as a matrix product, $W_{ji}x_i$?

$$e^{1+W_{ji}x_i} = e \cdot e^{W_{ji}x_i}$$

If we write W and x in the log domain first, we have:

$$e \cdot e^{\log(W_{ji})\log(x_i)} = e \cdot (W_{ji} + x_i)$$

DerivedModel: Ergodic Exploration via Adaptive Bias-Variance Control

BasicModel implements a novel approach to neural network optimization that decouples **what** the network learns (weights W) from **how much it explores** (a scalar α governing the bias-variance tradeoff). Two fundamentally different algorithms operate in tandem:

Component	Algorithm	What it does
Weights W	Standard Adam	Gradient descent on the loss landscape
Tradeoff α	Gradient Energy Sensor	Measures loss-surface curvature to set exploration level

The key insight: α **is not trained by gradient descent.** It is a *sensor* that reads the gradient energy flowing through the temperature parameter and converts it into an exploration policy.

---

1. The Ergodic Weights

In this model, weights are not treated as fixed constants. Each layer instead uses an effective weight matrix sampled from a locally specified ergodic distribution, with the learned tensor and the stochastic tensor defining the current mixture:

$$W_{\text{eff}} = \alpha \cdot W + (1 - \alpha) \cdot \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I)$$

So the layer never acts on a bare, timeless weight matrix; it acts on the current sample W_{eff}, while W stores the learned structure that increasingly shapes that local distribution as α rises. This perspective is adjacent to the broader literature on stochastic sampling in learning, including Monte Carlo methods, stochastic approximation, and noise-injected views of neural network optimization.

We define two derived quantities from the single scalar α:

$$\text{bias} = \alpha, \qquad \text{temp} = 1 - \alpha$$

This is a **direct encoding of the bias-variance tradeoff**:

- $\alpha = 0$: Pure exploration. $W_{\text{eff}} = \varepsilon$. The network is random noise.
- $\alpha = 1$: Pure exploitation. $W_{\text{eff}} = W$. The network uses only learned weights.
- $0 < \alpha < 1$: Interpolation between learned structure and stochastic exploration.

Training begins at $\alpha = 0$ (full exploration) and the system autonomously transitions toward exploitation as the loss landscape flattens.

---

2. Why Standard Adam Fails on α

The temperature parameter $T = 1 - \alpha$ receives gradients via:

$$\frac{\partial \mathcal{L}}{\partial T} = \sum_{i,j} \frac{\partial \mathcal{L}}{\partial (W_{\text{eff}})_{ij}} \cdot \varepsilon_{ij}$$

Because ε is **re-sampled every forward pass**, this gradient is **zero-mean**:

$$\mathbb{E}\left[\frac{\partial \mathcal{L}}{\partial T}\right] = \sum_{i,j} \frac{\partial \mathcal{L}}{\partial (W_{\text{eff}})_{ij}} \cdot \underbrace{\mathbb{E}[\varepsilon_{ij}]}_{=0} = 0$$

Standard Adam maintains a first moment (running mean) and a second moment (running variance), then computes the update as $m/\sqrt{v}$. For a zero-mean gradient:

$$m_t \approx 0 \quad \Longrightarrow \quad \frac{m_t}{\sqrt{v_t}} \approx \frac{0}{\sqrt{v_t}} = 0$$

Adam produces no update. The first moment kills the signal. This is correct behavior for Adam — it correctly identifies that there is no consistent gradient direction — but it means Adam cannot be used to tune α.

---

3. The Gradient Energy Sensor

3.1 Modified Second-Moment Estimator

We discard the first moment entirely and use only the second moment, but as **output** rather than normalizer:

$$v_t = \beta \cdot v_{t-1} + (1 - \beta) \cdot g_t^2$$

where $g_t = \partial \mathcal{L} / \partial T$ is the temperature gradient at step t, and $\beta = 0.999$ is the EMA decay rate.

3.2 Bias Correction

Identical to Adam's bias correction. Since $v_0 = 0$, early estimates are biased toward zero:

$$\hat{v}_t = \frac{v_t}{1 - \beta^t}$$

3.3 What $\hat{v}_t$ Measures

The second moment of a zero-mean random variable is its variance:

$$\mathbb{E}[g_t^2] = \text{Var}(g_t) = \sum_{i,j} \left(\frac{\partial \mathcal{L}}{\partial (W_{\text{eff}})_{ij}} \right)^2 = \left\| \frac{\partial \mathcal{L}}{\partial W_{\text{eff}}} \right\|_F^2$$

This is the **squared Frobenius norm** of the loss gradient with respect to the effective weights — a scalar measure of how much the loss surface cares about weight perturbations. We call this the **gradient energy**.

Gradient energy	Meaning	Desired behavior
High $\hat{v}_t$	Loss is sensitive to weight changes	Keep exploring (low α)
Low $\hat{v}_t$	Loss surface is flat	Exploit learned weights (high α)

3.4 The Alpha Update Rule

$$\alpha_t = \frac{1}{1 + \tau \cdot \sqrt{\hat{v}_t}}$$

where τ (`global_temp`) is a tunable sensitivity knob.

Properties:

- $\sqrt{\hat{v}_t} \to 0 \implies \alpha_t \to 1$ (exploit when gradients vanish)
- $\sqrt{\hat{v}_t} \to \infty \implies \alpha_t \to 0$ (explore when gradients are large)
- $\alpha_t \in (0, 1)$ always — the sigmoid-like shape prevents saturation
- τ controls the transition speed: large τ means more exploration at equivalent gradient energy

After computing α_t, the bias and temperature are derived:

$$\text{bias}_t = \alpha_t, \qquad T_t = 1 - \alpha_t$$

3.5 Layer-Local Certainty

The scalar α_t is a **global layer policy**, but individual output neurons do not need to converge at the same rate. Each ergodic layer therefore maintains a certainty vector

$$c_t \in [0, 1]^{n_{\text{out}}}$$

whose entries are tuned from two separate signals kept outside the core ergodic update: a forward-pass activation statistic and the historical gradient energy of the learned per-output parameters in that layer.

The forward signal is a bounded summary of activation magnitude:

$$f_{t,j} = \tanh(\mathbb{E}[|y_{t,j}|])$$

The gradient signal is:

$$v_{t,j}^{(c)} = \beta_c v_{t-1,j}^{(c)} + (1 - \beta_c) \cdot g_{t,j}^2$$

$$\hat{v}_{t,j}^{(c)} = \frac{v_{t,j}^{(c)}}{1 - \beta_c^t}, \qquad g_{t,j}^{(c)} = \frac{1}{1 + \tau_c \sqrt{\hat{v}_{t,j}^{(c)}}}$$

The final local certainty is a blend of the forward and gradient views:

$$c_{t,j} = \lambda_f f_{t,j} + \lambda_g g_{t,j}^{(c)}, \qquad \lambda_f + \lambda_g = 1$$

This yields neuron-wise bias/variance coefficients:

$$\text{bias}_{t,j}^{\text{local}} = c_{t,j} \cdot \text{bias}_t$$

$$T_{t,j}^{\text{local}} = T_t + (1 - c_{t,j}) \cdot \text{bias}_t$$

so that $\text{bias}_{t,j}^{\text{local}} + T_{t,j}^{\text{local}} = 1$ whenever dropout is inactive. High-certainty outputs stabilize early and lean on learned weights; low-certainty outputs keep more noise and continue exploring. This lets earlier or easier features settle before later or less reliable ones without changing the global ergodic algorithm.

---

4. Comparison: Adam vs. Gradient Energy Sensor

Property	Adam (for W)	Gradient Energy Sensor (for α)
Type	Optimizer (gradient descent)	Sensor (measurement)
First moment m_t	Yes — tracks gradient direction	No — direction is zero-mean, useless
Second moment v_t	Yes — normalizes step size	Yes — **is the output**
Role of $\sqrt{v_t}$	Denominator (adaptive learning rate)	Numerator (gradient energy estimate)
Update rule	$W \leftarrow W - \eta \cdot m/\sqrt{v}$	$\alpha \leftarrow 1/(1 + \tau\sqrt{\hat{v}})$
Descent?	Yes — follows gradient downhill	No — maps energy to a policy
Zero-mean safe?	No — produces $0/\sqrt{v} = 0$	Yes — by design

---

5. Derivation: Why Zero-Mean Implies Drop m_t

Let $g_t = \nabla_T \mathcal{L}$ be the temperature gradient. Since noise ε is i.i.d. each step:

$$g_t = \sum_{i,j} \frac{\partial \mathcal{L}}{\partial (W_{\text{eff}})_{ij}} \cdot \varepsilon_{ij}^{(t)}$$

The first moment EMA is:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

Taking expectations:

$$\mathbb{E}[m_t] = \beta_1 \mathbb{E}[m_{t-1}] + (1 - \beta_1) \underbrace{\mathbb{E}[g_t]}_{=0} = \beta_1 \mathbb{E}[m_{t-1}]$$

By induction: $\mathbb{E}[m_t] = \beta_1^t \mathbb{E}[m_0] = 0$. The first moment converges exponentially to zero. It carries **no information** about the loss landscape — only noise from finite sampling. Keeping it would add variance to the estimator without adding signal.

The second moment, however, converges to a meaningful quantity:

$$\mathbb{E}[v_t] \to \mathbb{E}[g_t^2] = \left\| \nabla_{W_{\text{eff}}} \mathcal{L} \right\|_F^2$$

This is exactly the gradient energy — the quantity we need.

---

6. Dropout via Bias

The `bias` and `temp` parameters are passed through the full layer stack to support **dropout regularization**:

$$\text{bias}_t^{(\text{drop})} = \begin{cases} 0 & \text{with probability } p \\ \alpha_t & \text{with probability } 1-p \end{cases}$$

When bias is dropped to zero, the layer acts as pure noise regardless of α. This is analogous to standard dropout but operates on the bias-variance tradeoff rather than individual activations. The temperature $T = 1 - \alpha$ remains unchanged so that the noise contribution is consistent.

Seen this way, dropout is just a schedule on bias. Ordinary Bernoulli dropout is the binary case in which the bias is either kept at α_t or forced to 0 for that step, while annealed dropout corresponds to increasing the expected bias over training so the layer spends less time in pure-noise mode and more time exploiting learned structure. More generally, nonzero temperature bakes regularization directly into the model because every forward pass perturbs the effective weights, discouraging brittle co-adaptation and rewarding structure that survives stochastic variation.

Every layer signature accepts `(bias, temp)` to support this:

```
def forward(self, x, bias=1.0, temp=0.0):
    W_eff = bias * W + temp * noise
    ...
```

---

7. The `global_temp` Sensitivity Knob

The parameter τ (`global_temp`) scales how responsive α is to gradient energy:

$$\alpha = \frac{1}{1 + \tau \cdot \sqrt{\hat{v}}}$$

τ	Effect
$\tau = 0$	$\alpha = 1$ always — pure exploitation, no exploration
$\tau \ll 1$	Aggressive exploitation — quickly converges to $\alpha \approx 1$
$\tau = 1$	Balanced — α tracks gradient energy on a natural scale
$\tau \gg 1$	Conservative — maintains high exploration even with moderate gradients

In practice, τ can itself be scheduled (e.g., annealed from high to low) to implement a coarse exploration-to-exploitation curriculum.

---

8. Initialization and Bootstrap

Problem: If we initialized at $\alpha = 1$ (pure exploitation), the noise term vanishes and the temperature gradient $\partial\mathcal{L}/\partial T = 0$. The sensor would read zero gradient energy and keep $\alpha = 1$ forever — an **exploitation trap**.

Solution: Initialize at $\alpha = 0$ (pure exploration):

```
self.alpha       = nn.Parameter(torch.tensor(0.0), requires_grad=False)
self.bias        = nn.Parameter(torch.tensor(0.0), requires_grad=False)
self.temperature = nn.Parameter(torch.tensor(1.0), requires_grad=True)
```

At $\alpha = 0$:

- $W_{\text{eff}} = \varepsilon$ — pure noise carries gradients through the network
- The temperature gradient is nonzero: $g_t = \sum_{ij}(\partial\mathcal{L}/\partial\varepsilon_{ij}) \cdot \varepsilon_{ij} \neq 0$
- The sensor measures initial gradient energy and begins tuning α upward
- As W learns useful structure, gradient energy decreases and α rises naturally

In this architecture, zero-initializing W is not only safe but often cleaner than random initialization. In an ordinary network, zero initialization is bad because it fails to break symmetry, but here symmetry is already broken by the sampled noise ε in W_{eff}. Since training begins at $\alpha = 0$, the learned weights do not contribute to the forward pass anyway, so random weight initialization only injects arbitrary structure into a phase that is meant to be pure exploration. Starting from $W = 0$ makes that exploratory regime honest and lets useful structure enter only when the bias toward learned weights increases.

---

9. Layer Architecture

All layers follow the effective-weight pattern with `(bias, temp)` signatures:

LinearLayer

$$y = x \cdot (\text{bias} \cdot W + \text{temp} \cdot \varepsilon_W) + \text{bias} \cdot b + \text{temp} \cdot \varepsilon_b$$

ReversibleLinearLayer (SVD decomposition)

$$W = U\Sigma V^\top$$

Each factor applies the tradeoff independently:

$$y = V \cdot \Sigma \cdot U^\top \cdot x$$

where each rotation and scaling layer uses `(bias, temp)` internally.

ReversibleRotationLayer

Parameterized by Givens angles θ_k:

$$\theta_k^{\text{eff}} = \text{bias} \cdot \theta_k + \text{temp} \cdot \varepsilon_k$$

ReversibleDiagonalLayer

Parameterized by log-singular-values λ_k:

$$\lambda_k^{\text{eff}} = \text{bias} \cdot \lambda_k + \text{temp} \cdot \varepsilon_k$$

---

10. Training Loop Integration

```
# In the training loop (Ergodic.py):

# 1. Standard Adam optimizer for W (excludes temperature)
optimizer = torch.optim.Adam(model.getParameters(), lr=lr)

# 2. Forward pass computes W_eff = bias*W + temp*noise
loss = model(x, y)
```

```
# 3. Backward pass: Adam gets dL/dW, temperature gets dL/dT
loss.backward()

# 4. Adam updates W
optimizer.step()

# 5. The sensor updates alpha, while local certainty blends forward activity and gradient history
model.paramUpdate()  # calls alpha_update() on each ErgodicLayer
```

Note that `getParameters()` **excludes** the temperature parameter:

```
def getParameters(self):
    params = [p for n, p in self.named_parameters() if n != "temperature"]
    return params
```

Temperature is excluded from Adam because Adam would produce zero updates on it (see Section 2). Instead, the gradient energy sensor reads `temperature.grad` directly.

---

11. Summary of the Full Algorithm

Initialize:

$$\alpha_0 = 0, \quad v_0 = 0, \quad t = 0, \quad \beta = 0.999$$

Each training step:

1. Sample $\varepsilon \sim \mathcal{N}(0, I)$
2. Compute $W_{\text{eff}} = \alpha_t W + (1 - \alpha_t)\varepsilon$
3. Forward pass: $\hat{y} = f(x; W_{\text{eff}})$
4. Compute loss $\mathcal{L}(\hat{y}, y)$
5. Backward pass: compute $\nabla_W \mathcal{L}$ and $g_t = \nabla_T \mathcal{L}$
6. **Adam** updates W: $W \leftarrow W - \eta \cdot \hat{m}_t / \sqrt{\hat{v}_t^{(W)}}$
7. **Sensor** updates α, and a separate certainty routine blends forward activity with gradient history:

$$v_t \leftarrow \beta \cdot v_{t-1} + (1 - \beta) \cdot g_t^2$$

$$\hat{v}_t = \frac{v_t}{1 - \beta^t}$$

$$\alpha_{t+1} = \frac{1}{1 + \tau \sqrt{\hat{v}_t}}$$

$$c_{t+1,j} = \frac{1}{1 + \tau_c \sqrt{\hat{v}_{t,j}^{(c)}}}$$

$$\text{bias}_{t+1,j}^{\text{local}} = c_{t+1,j} \cdot \text{bias}_{t+1}, \qquad T_{t+1,j}^{\text{local}} = T_{t+1} + (1 - c_{t+1,j}) \cdot \text{bias}_{t+1}$$

8. Zero temperature gradient: $g_t \leftarrow 0$